LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS

NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram Kelas dan Relasi Antar Kelas

Diagram kelas adalah diagram yang digunakan untuk memodelkan kelas-kelas dalam sebuah sistem beserta hubungan antar kelas tersebut. Setiap kelas digambarkan sebagai kotak yang terdiri dari tiga bagian: nama kelas pada baris pertama, atribut-atribut pada bagian kedua, dan method-method pada bagian ketiga. Hak akses pada atribut dan method ditandai dengan simbol tertentu, yaitu tanda plus (+) untuk public, tanda minus (-) untuk private, dan tanda pagar (#) untuk protected. Hubungan antar kelas digambarkan dengan garis yang menghubungkan dua kotak kelas, dan bentuk ujung garisnya menunjukkan jenis relasi yang terjadi.

Dalam Object Oriented Programming, kelas-kelas yang terbentuk dapat saling berhubungan sesuai dengan kondisi kelas-kelas yang bersangkutan. Jenis hubungan yang dapat terjadi antara lain asosiasi, agregasi, komposisi, serta generalisasi dan spesialisasi yang berkaitan dengan pewarisan. Pemilihan jenis relasi yang tepat penting agar rancangan program mencerminkan kondisi dunia nyata yang dimodelkan, khususnya terkait seberapa kuat keterikatan antar objek dan bagaimana siklus hidup masing-masing objek.

Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, maka hubungan tersebut diwakili oleh asosiasi. Pada asosiasi, objek dari satu kelas tidak dijadikan atribut dari kelas lainnya, melainkan hanya digunakan ketika sebuah method dipanggil, biasanya dengan cara diterima sebagai parameter. Sebagai contoh, kelas Dosen memiliki method giveScore() yang menerima objek Mahasiswa sebagai parameter untuk memberikan nilai, namun objek Mahasiswa tersebut tidak disimpan sebagai atribut kelas Dosen. Dengan demikian, kedua objek saling menggunakan tetapi tidak saling memiliki. Pada diagram kelas, asosiasi digambarkan dengan garis lurus sederhana di antara dua kelas.

Agregasi

Agregasi merupakan hubungan antara dua kelas yang lebih ketat dibandingkan asosiasi, karena terdapat hubungan bagian (part) dan keseluruhan (whole). Pada agregasi, kelas yang menjadi bagian merupakan elemen dari kelas keseluruhan, tetapi keduanya tetap dapat diciptakan dan berdiri sendiri-sendiri. Contohnya adalah hubungan antara kelas Jurusan dan kelas Mahasiswa, di mana Jurusan memiliki atribut berupa array bertipe Mahasiswa. Objek Mahasiswa dibuat di luar kelas Jurusan, lalu dimasukkan melalui method seperti AddMahasiswa(). Apabila objek Jurusan dihapus, objek Mahasiswa

secara konsep tetap ada. Agregasi digambarkan dengan garis yang ujungnya berbentuk belah ketupat kosong pada sisi kelas keseluruhan.

Komposisi

Komposisi merupakan bentuk khusus dari agregasi dengan hubungan yang lebih kuat, karena kelas yang menjadi bagian (part) tidak dapat ada tanpa adanya kelas yang menjadi keseluruhan (whole). Siklus hidup objek bagian terikat penuh pada objek keseluruhannya, sehingga ketika objek keseluruhan dihancurkan, objek bagiannya ikut hilang. Contohnya adalah kelas Komputer yang tersusun atas CPU, Monitor, dan Mouse. Pada implementasinya, objek-objek bagian tersebut baru dibuat pada saat objek Komputer diinstansiasi, yaitu di dalam constructor kelas keseluruhan, bukan dibuat dari luar. Komposisi digambarkan dengan garis yang ujungnya berbentuk belah ketupat terisi (hitam) pada sisi kelas keseluruhan.

Enkapsulasi

Dalam penerapan ketiga relasi tersebut, prinsip enkapsulasi tetap dijaga dengan mendeklarasikan atribut sebagai private dan menyediakan akses melalui method getter dan setter. Kata kunci `this` digunakan untuk membedakan atribut milik objek dengan parameter yang bernama sama, khususnya pada constructor. Dengan begitu, data di dalam objek tidak dapat diakses atau diubah secara langsung dari luar kelasnya.

B. Guided

1. Asosiasi

Mahasiswa.java

```
package guided1.asosiasi;


public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;


    public void setNim(String nim) {
        this.nim = nim;
```

```
    }

    public String getNim() {
        return this.nim;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return this.nama;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public int getNilai() {
        return this.nilai;
    }
}
```

Dosen.java

```
package guided1.asosiasi;

public class Dosen {
    private String kodeDosen;
```

```
    private String namaDosen;
    private int nilai;

    //Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
    s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
    return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("103132400038");
        m.setNama("Humairah Nurhamidah");
        Dosen d = new Dosen();
        d.giveScore(m, 100);
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ mavenproject1 ---
Nim :103132400038
Nama :Humairah Nurhamidah
Nilai :100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Agregasi

Mahasiswa.java

```
package guided2.Agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String GetNIM() {
        return (NIM);
    }

    public String GetNama() {
        return (Nama);
    }

    //Setter
    public void SetNim(String NIM){
        this.NIM = NIM;
    }

    public void SetNama(String Nama){
        this.Nama = Nama;
    }
}
```

Jurusan

```
package guided2.Agregasi;
```

```
public class Jurusan {

    private static int JumlahMahasiswa = 0;
    private String KodeJurusan, NamaJurusan;
    private Mahasiswa[] DaftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan) {
        this.KodeJurusan = kodeJurusan;
        this.NamaJurusan = namaJurusan;
    }

    // Behaviour
    public void AddMahasiswa (Mahasiswa m) {
        this.DaftarMahasiswa[JumlahMahasiswa] = m;
        JumlahMahasiswa++;
    }

    public void DisplayMahasiswa() {
        System.out.println("Kode Jurusan : " + this.KodeJurusan);
        System.out.println("Nama Jurusan : " + this.NamaJurusan);

        System.out.println("Daftar Mahasiswa :");

        for (int i = 0; i < JumlahMahasiswa; i++) {
            System.out.println((i + 1) + ". " + DaftarMahasiswa[i].GetNIM() +
" " + DaftarMahasiswa[i].GetNama());
        }
    }

    // Getter
    public String GetKodeJurusan() {
        return KodeJurusan;
    }

    public String GetNamaJurusan() {
        return NamaJurusan;
    }

    public static int GetJumlahMahasiswa() {
        return JumlahMahasiswa;
    }

    public Mahasiswa[] GetDaftarMahasiswa() {
        return DaftarMahasiswa;
```

```
    }

    // Setter
    public void SetKodeJurusan(String KodeJurusan) {
        this.KodeJurusan = KodeJurusan;
    }

    public void SetNamaJurusan(String NamaJurusan) {
        this.NamaJurusan = NamaJurusan;
    }
}
```

Main.java

```
package guided2.Agregasi;

public class Main {
    public static void main(String[] args) {
        runAsosiasi();
    }

    public static void runAgregasi() {}

    public static void runAsosiasi() {
        Mahasiswa m1 = new Mahasiswa("103132400038", "Humaira");
        Mahasiswa m2 = new Mahasiswa("103132400016", "Andita");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.AddMahasiswa(m1);
        prodi.AddMahasiswa(m2);

        prodi.DisplayMahasiswa();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ mavenproject1 ---
Kode Jurusan : SD
Nama Jurusan : Sains Data
Daftar Mahasiswa :
1. 103132400038 Humaira
2. 103132400016 Andita
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Komposisi

Processor.java

```
package guided3.Komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Monitor.java

```
package guided3.Komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
```

```
    }
}
```

Mouse.java

```
package guided3.Komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Komputer.java

```
package guided3.Komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor,
String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

public void displaySpecKomputer() {
    System.out.println("=== SPESIFIKASI KOMPUTER ===");
    this.processor.displaySpec();
    this.monitor.DisplaySpec();
    this.mouse.displaySpec();
}
}
```

Main.java

```
package guided3.Komposisi;

import guided2.Agregasi.*;
import guided3.Komposisi.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }


    public static void runGuided1(){}

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103132400038", "Humairah");
        Mahasiswa m2 = new Mahasiswa("103132400016", "Andita");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.AddMahasiswa(m1);
        prodi.AddMahasiswa(m2);
        prodi.DisplayMahasiswa();

    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ mavenproject1 ---
Kode Jurusan : SD
Nama Jurusan : Sains Data
Daftar Mahasiswa :
1. 103132400038 Humairah
2. 103132400016 Andita
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

   Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe. Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

   Kebutuhan Tugas :

   1. Membuat dua class: RincianHarga dan ItemPesanan. (Komposisi)
   2. Membuat class Pesanan (Agregasi)
   3. Membuat class Kasir (Asosiasi)
   4. Membuat class Main untuk pengujian

   Jawaban :

   RincianHarga

```
package Unguided;

public class RincianHarga {

    private double hargaDasar;

    private double pajak;


    public RincianHarga(double hargaDasar) {

        this.hargaDasar = hargaDasar;
```

```
        this.pajak = hargaDasar * 0.10; // pajak 10% dari harga dasar
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

Kasir

```
package Unguided;
public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // ASOSIASI: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai
field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
```

```
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("================================");
    }
}
```

Pesanan

```
package Unguided;
public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // AGREGASI
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak
dapat ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
```

```
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
        System.out.println();
    }

    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

ItemPesanan

```
package Unguided;
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // KOMPOSISI

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // RincianHarga dibuat di dalam constructor, bukan di main
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

Main

```
package Unguided;
public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        // Item dibuat independen di luar Pesanan (agregasi)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
- ==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini terdiri dari lima class (RincianHarga, ItemPesanan, Pesanan, Kasir, dan Main) yang menerapkan tiga jenis relasi antar kelas. Semua field dibuat private dan

diakses lewat getter, sedangkan constructor memakai this karena nama parameternya sama dengan nama field.

Relasi **komposisi** ada pada ItemPesanan dan RincianHarga. Objek RincianHarga dibuat dengan new di dalam constructor ItemPesanan, bukan di main, dan pajak 10% dihitung otomatis dari harga dasar. Karena rincian harga tidak bisa ada tanpa item pesanannya, jika ItemPesanan dihapus maka RincianHarga ikut hilang.

Relasi **agregasi** ada pada Pesanan dan ItemPesanan. Objek ItemPesanan dibuat lebih dulu di main, lalu dimasukkan ke array daftarItem milik Pesanan lewat tambahItem(). Karena itu, item tetap ada meskipun pesanannya dibatalkan.

Relasi **asosiasi** ada pada Kasir dan Pesanan. Kasir tidak menyimpan Pesanan sebagai field, tetapi menerimanya sebagai parameter pada prosesPembayaran(). Method ini menjumlahkan subtotal dan pajak seluruh item, lalu mencetak struk.

Pada Main, dibuat satu kasir (Budi), tiga item, dan satu pesanan untuk meja 4. Hasilnya, total subtotal Rp 65.000, total pajak Rp 6.500, dan total bayar Rp 71.500, sesuai dengan output yang diminta soal.

D. Kesimpulan

Berdasarkan praktikum Modul 5 tentang relasi antar kelas, dapat disimpulkan bahwa kelas-kelas dalam pemrograman berorientasi objek tidak berdiri sendiri, melainkan saling berhubungan sesuai kondisi nyata yang dimodelkan. Tiga jenis relasi yang dipelajari, yaitu asosiasi, agregasi, dan komposisi, dibedakan berdasarkan seberapa kuat keterikatan antar objek dan bagaimana siklus hidupnya.

Asosiasi adalah hubungan paling longgar, di mana satu objek hanya menggunakan objek lain melalui parameter method tanpa menyimpannya sebagai atribut. Agregasi lebih ketat karena ada hubungan bagian dan keseluruhan, tetapi objek bagian tetap bisa dibuat dan berdiri sendiri di luar objek wadahnya. Komposisi adalah bentuk agregasi yang paling kuat, karena objek bagian dibuat di dalam constructor objek keseluruhan dan tidak dapat ada tanpanya, sehingga ikut hilang ketika objek keseluruhan dihapus.

Pada studi kasus sistem pesanan cafe, ketiga relasi tersebut berhasil diterapkan: ItemPesanan dan RincianHarga berelasi komposisi, Pesanan dan ItemPesanan berelasi agregasi, serta Kasir dan Pesanan berelasi asosiasi. Penerapan enkapsulasi lewat atribut private dan getter juga menjaga agar data tidak diubah langsung dari luar class. Dengan memahami perbedaan ketiga relasi ini, praktikan dapat merancang struktur program yang lebih terorganisir, mudah dipahami, dan sesuai dengan kondisi sebenarnya.